Universidad autónoma de san Luis potosí

Facultad de ingeniería

Reporte:

Uno

Integrantes:

- Elzy Michelle Cruz Francisco @Elzymichelle
- Banda Llanas Ángel @Llanazzz
- Benavente Nájera Alejandro @aleeeebn

Carrera: Ingeniería en Sistemas Inteligentes

Materia: interfaces graficas con aplicaciones

Fecha: 11 de octubre de 2026

Este proyecto es una versión gráfica del juego UNO, programada en C# con Windows Forms, que sigue las reglas oficiales, corre en una sola computadora para tres jugadores y guarda sus datos en una base de datos MySQL a través de una API en Python.

1. Descripción general del proyecto

El objetivo fue programar el UNO completo, con interfaz gráfica basada en imágenes, donde una sola persona controla a todos los jugadores en la misma computadora. Como el equipo es de tres integrantes, cada partida se juega con tres jugadores.

El juego incluye un menú inicial para capturar los nombres, el barajeo y reparto de cartas, la validación de cada jugada según las reglas oficiales, el robo de cartas, pasar turno, decir UNO y acusar a quien no lo dijo. Además, se juega por rondas con sistema de puntos: gana la partida el primer jugador que llega a 500 puntos.

El juego se conecta a una base de datos que guarda tres cosas, tal como lo pide la actividad:

- Los jugadores (nombre o id).
- El historial de partidas (ganadas y perdidas por jugador).
- El log del juego, con todos los movimientos realizados.

El proyecto está completo y funcional. Para lograrlo, el trabajo se dividió en tres áreas: la interfaz gráfica, la lógica del juego y la base de datos con su API.

2. Tecnologías y arquitectura del sistema

El sistema se divide en tres capas que se comunican en orden: el juego en C#, una API en Python y la base de datos MySQL.

Capa	Tecnología	Función
Juego (cliente)	C# con Windows Forms	Interfaz gráfica, reglas y lógica de la partida
Comunicación	HttpClient + Newtonsoft.Json (clase ApiCliente)	Envía las peticiones del juego a la API
API	Python con FastAPI	Recibe las peticiones y consulta o guarda en la base de datos
Base de datos	MySQL (diseño en Workbench)	Guarda jugadores, partidas y log de movimientos
Control de versiones	Git y GitHub	Ramas por integrante y Pull Requests hacia main

Flujo de una jugada: el jugador hace clic en una carta en Form2, el juego valida la jugada con las clases de lógica, la interfaz se actualiza y ApiCliente manda el movimiento a la API, que lo inserta en la tabla log_movimientos.

3. División de tareas entre los integrantes

Los tres integrantes participaron en actividades de programación. La tabla resume el reparto; el detalle de cada quien viene después.

Integrante	Área	Archivos / componentes principales
Elzy	Interfaz gráfica y reporte	Menu.cs, Form2.cs, ImagenesCartas.cs, diseño de las imágenes de las cartas
Alejandro	Lógica del juego e implementación de clases	Carta, CartasJuego, Jugador, Mazo, Partida; partes del diseño
Llanas	Base de datos, API y conexión	Script uno_db.sql, API FastAPI, ApiCliente, conexión en Form2

Elzy: interfaz gráfica y reporte. Todo lo que el jugador ve y toca: los menús, la mesa de juego y las imágenes de las cartas. Redacción del reporte.

Tarea	Detalle	Archivo o componente
Menú inicial	Pantalla de entrada del juego, desde la que se pasa a la partida.	Menu.cs
Pantalla principal de juego	Interfaz final de la partida, diseñada completamente por código. Muestra a los tres jugadores (arriba, a la izquierda y a la derecha), el mazo y la carta central en el centro, y la mano del jugador en turno abajo. Reemplazó al Form1 de pruebas con botones, que ya no se usa.	Form2.cs
Carga de imágenes de cartas	Clase que relaciona cada carta del juego con su imagen para que la interfaz la dibuje en pantalla.	ImagenesCartas.cs
Diseño gráfico de las cartas	Diseño propio de las imágenes de las cartas, con estilo tierno, colores pastel y tonos cálidos.	Carpeta de imágenes del proyecto
Reporte	Redacción del documento: descripción, división de tareas, base de datos, commits, ramas y problemas.	Reporte

Form2 es el punto donde se juntan las tres partes del trabajo: usa las clases de lógica de Alejandro para saber qué jugadas son válidas y las llamadas a la API de Llanas para guardar cada movimiento.

Alejandro: lógica del juego y clases. programó el corazón del juego: las clases que representan las cartas, los jugadores y la partida, y todas las reglas que se comprueban en cada turno. También implementó varias piezas visuales del juego.

Clases y métodos

Clase	Qué hace
Carta	Representa una carta individual con sus características.
CartasJuego	Define el conjunto de cartas que forma el juego.
Mazo	Maneja el mazo: barajear, repartir y robar cartas.
Jugador	Guarda la mano y los datos de cada jugador.
Partida	Controla el desarrollo del juego: turnos, comprobaciones, reglas, puntos y rondas.

Sistema del juego

- Barajear y repartir las cartas al inicio de cada ronda.
- Comprobaciones al jugar cartas, para que solo se pueda tirar una carta válida.
- Aplicación de las reglas del UNO original.
- Sistema de puntos y rondas, hasta que un jugador llega a 500 puntos.
- Sistema para robar, pasar y acusar.
- Contadores de victorias.

Partes del diseño

- Rueda de dirección de turno.
- Menú de nombres de los jugadores.
- Pantalla de victoria.
- Botón de pasar y botón de acusar.
- Assets del juego.

Llanas: base de datos, API y conexión. Se encargó de que el juego guarde su información: diseñó la base de datos, construyó la API que la conecta con el juego y programó la conexión desde C#. También administró la integración del trabajo en Git.

Base de datos (MySQL)

- Diseñó la base y el diagrama entidad-relación con 4 tablas: jugadores, partidas, partida_jugadores y log_movimientos, con sus llaves primarias y foráneas.
- Generó el script uno_db.sql y el diagrama (diagrama_er.png), ambos en la carpeta database del repositorio.
- Decidió que el historial de ganadas y perdidas no se guarda con contadores, sino que se calcula a partir de las partidas guardadas, para que los totales siempre coincidan.

API (Python con FastAPI)

- Creó la API que conecta el juego con la base de datos, con los endpoints GET /jugadores, POST /jugadores, POST /partidas, POST /partidas/{id}/terminar, POST /movimientos y GET /historial.
- Dejó las credenciales de MySQL en un archivo .env

Conexión del juego (C#)

- Creó la clase ApiCliente, que usa HttpClient y Newtonsoft.Json para llamar a la API desde Windows Forms.
- Conectó Form2 para que guarde los jugadores por nombre, la partida, cada movimiento (jugar carta, robar, pasar, elegir color, decir UNO y penalizaciones) y al campeón al terminar.
- Hizo que cada ronda quede en el log como ganar_ronda con sus puntos, y que solo quien llega a 500 cierre la partida como ganador.

Git

- Trabajó en la rama BD-API e integró sus cambios a main con Pull Requests.
- Mantuvo el .gitignore (entorno virtual, .env, respaldos de Workbench y paquetes de NuGet).
- Resolvió los problemas de integración descritos en la sección 9.

4. Reglas implementadas y funcionamiento del juego

El juego sigue las reglas oficiales del UNO. Se juega por rondas y la partida termina cuando un jugador acumula 500 puntos.

Flujo general

1. En el menú se capturan los nombres de los tres jugadores.
2. Se crea la partida, se baraja el mazo y se reparten las cartas.
3. Se voltea una carta inicial y comienza la ronda; la rueda de dirección indica de quién es el turno y hacia dónde va el juego.
4. En su turno, el jugador juega una carta válida, o roba una carta, o pasa turno.
5. Quien queda con una sola carta debe decir UNO; si no lo dice, otro jugador puede acusarlo y se aplica una penalización.
6. Quien se queda sin cartas gana la ronda y recibe puntos. Se juegan rondas hasta que alguien llega a 500 puntos.
7. Aparece la pantalla de victoria y el resultado se guarda en la base de datos.

Componentes de la lógica

Clase	Responsabilidad
Carta	Representa una carta individual (color, tipo y valor)
CartasJuego	Define el conjunto de cartas del juego
Mazo	Barajea, reparte y maneja el robo de cartas
Jugador	Guarda la mano y los datos de cada jugador
Partida	Controla turnos, comprobaciones de jugadas, reglas, puntos y rondas

Elementos de la interfaz

- Menú de nombres para capturar a los jugadores.
- Mesa de juego con los jugadores arriba y a los lados, el mazo y la carta central, y la mano abajo.
- Rueda de dirección de turno.
- Botones de pasar y de acusar.
- Elección de color al jugar cartas especiales.
- Pantalla de victoria y contadores de victorias.

5. Base de datos y diagrama entidad-relación

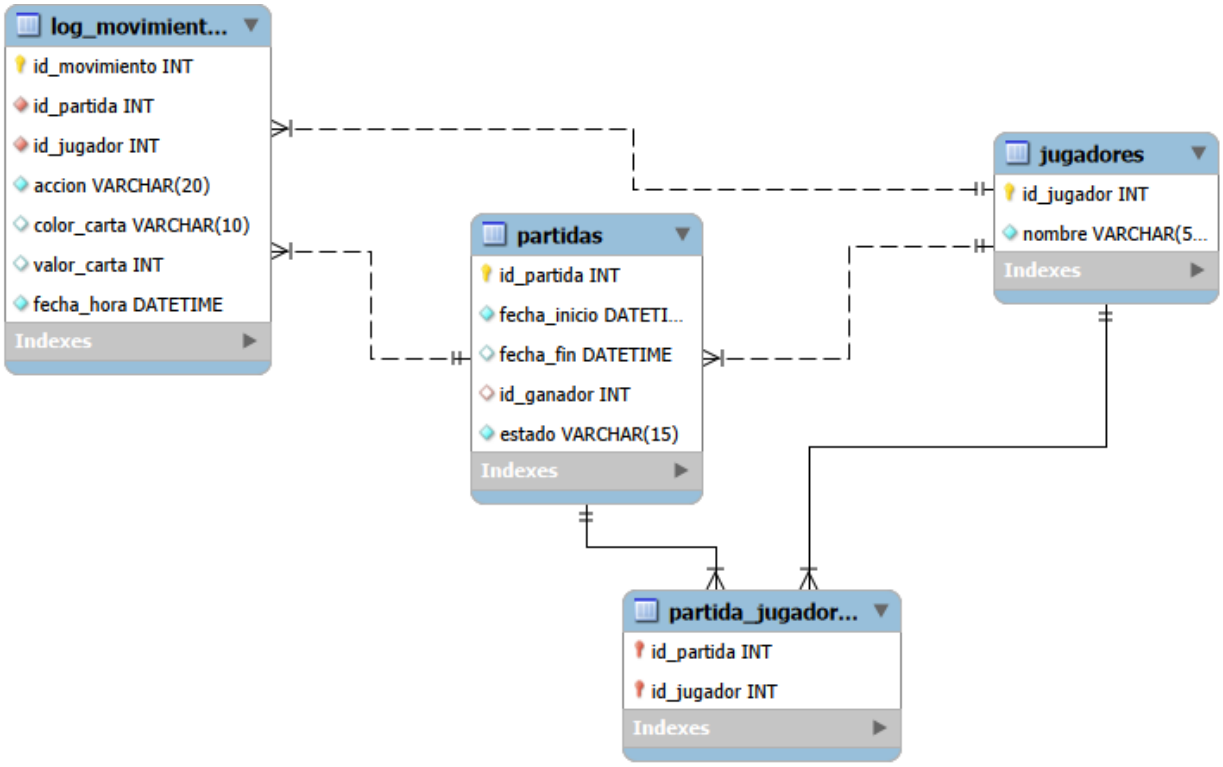
La base de datos está en MySQL y tiene 4 tablas con llaves primarias y foráneas. El script para crearla es uno_db.sql, y el diagrama está en la carpeta database del repositorio como diagrama_er.png.

Tabla	Qué guarda
jugadores	Los jugadores, identificados por nombre o id
partidas	Cada partida jugada
partida_jugadores	Qué jugadores participaron en cada partida y cómo les fue (relación entre jugadores y partidas)
log_movimientos	Todos los movimientos hechos en el juego, en orden

Decisión de diseño: el historial de ganadas y perdidas no se guarda con contadores. Se calcula a partir de las partidas guardadas, así los totales siempre coinciden con lo que realmente se jugó.

Qué es una partida: al cambiar el juego a rondas se redefinió el concepto. Una partida es el juego completo hasta llegar a 500 puntos. Cada ronda queda en el log como ganar_ronda con sus puntos, y solo el jugador que llega a 500 cierra la partida como ganador.

Diagrama entidad-relación:



Cada fila de partida_jugadores une a un jugador con una partida, y cada movimiento de log_movimientos queda ligado a la partida y al jugador que lo hizo.

El diagrama completo del equipo está en el archivo diagrama_er.png, dentro de la carpeta database del repositorio.

6. API y conexión del juego con la base de datos

El juego no habla directo con MySQL: lo hace a través de una API en Python con FastAPI. Esto separa la interfaz de la base de datos y evita poner credenciales dentro del juego.

Endpoint	Para qué sirve
GET /jugadores	Consulta los jugadores registrados
POST /jugadores	Registra un jugador por nombre
POST /partidas	Crea una partida nueva
POST /partidas/{id}/terminar	Cierra la partida y registra al campeón
POST /movimientos	Guarda un movimiento en el log
GET /historial	Devuelve el historial de ganadas y perdidas por jugador

Conexión desde C#

La clase ApiCliente usa HttpClient y Newtonsoft.Json para llamar a la API desde Windows Forms. Form2 la usa para guardar:

- Los jugadores, por nombre.
- La partida.
- Cada movimiento: jugar carta, robar, pasar, elegir color, decir UNO y penalizaciones.
- Al campeón, cuando termina la partida.

Cada ronda queda en el log como ganar_ronda con sus puntos. Para que los movimientos queden en el orden correcto, las peticiones salen por una cola que manda cada una después de la anterior.

7. Listado de commits

Fecha	Commit	Autor	Mensaje	Relación con las tareas
2026-10-11	e786f67	aleeeebn	Merge pull request #20 from aleeeebn/Alex	Integración de la rama Alex a main
2026-10-11	7750084	aleeeebn	Mensaje de comprobacion al salir	Interfaz: confirmación al salir
2026-10-11	f74f28f	aleeeebn	Implementacion de los puntos debajo del nombre	Interfaz: puntos de cada jugador
2026-10-11	f888c74	aleeeebn	Deshabilitación de botones al haber una carta de cambio de color	Reglas: cambio de color
2026-10-11	a713d7f	aleeeebn	Correcciones de bug con el botón UNO	Reglas: botón UNO
2026-10-10	aad4c80	Llanazzz	Merge pull request #19 from aleeeebn/BD-API	Integración de la rama BD-API a main
2026-10-10	0c44838	Llanazzz	Reconecto Form2 con la API, partida por juego y rondas en el log	Conexión de Form2 con la API; rondas en el log
2026-10-10	57a91c7	aleeeebn	Merge pull request #18 from aleeeebn/Alex	Integración de la rama Alex a main
2026-10-10	93faab6	aleeeebn	Menú para guardar nombres	Menú de nombres de los jugadores
2026-10-10	7bcbc6b	aleeeebn	Merge branch 'main' into Alex	Actualización de la rama Alex con main
2026-10-10	476583a	aleeeebn	Caché (ignorar)	Git: caché fuera del seguimiento

2026-10-10	2d4962b	aleeeebn	Merge remote-tracking branch 'origin/main' into ElzyMich	Actualización de la rama ElzyMich con main
2026-10-10	f75e851	aleeeebn	.	Ajuste menor
2026-10-10	f355130	aleeeebn	Merge pull request #16 from aleeeebn/Alex	Integración de la rama Alex a main
2026-10-10	7826013	aleeeebn	Merge branch 'main' into Alex	Actualización de la rama Alex con main
2026-10-10	864085d	aleeeebn	Caché	Git: caché
2026-10-10	51fe772	aleeeebn	Cambios en el +4, donde este solo se puede poner cuando no se tiene el color o simbolo de la carta en descarte	Reglas: carta +4
2026-10-10	8720011	aleeeebn	Modificación del juego. El juego ahora se basa como en las reglas oficiales las cuales dicen que un jugador gana hasta que sea el primero en alcanzar 500 puntos, estos se miden al final de cada ronda en base a las carrtas restantes de los demás	Reglas: rondas y ganador a 500 puntos
2026-10-10	c7686f8	Elzymichelle	menu	Interfaz: menú
2026-10-10	6a31c51	Elzymichelle	menu	Interfaz: menú
2026-10-10	4356b88	aleeeebn	Cambios al jugar un +4 como ultima carta para contar los puntos completos	Puntos: +4 como última carta
2026-10-10	f5de750	aleeeebn	Reacomodo del botón Acusar	Interfaz: botón Acusar
2026-10-10	a4b820c	aleeeebn	Implementación de botón para acusar el no haber puesto UNO	Reglas: acusar por no decir UNO
2026-10-10	2f2b127	Llanazzz	Merge pull request #14 from aleeeebn/BD-API	Integración de la rama BD-API a main
2026-10-10	a99d2c7	Llanazzz	Conexion Api con Forms2, ademas de corregir metodos utilizados para la conexion	Conexión de Form2 con la API
2026-10-10	11989a8	Llanazzz	Metodos para conectar Form2 con la API	Conexión de Form2 con la API
2026-10-10	9fddd56	Llanazzz	Agrego metodo para registrar las jugadas en ApiCliente	ApiCliente: registro de jugadas
2026-10-10	bf644d2	Llanazzz	Merge pull request #13 from aleeeebn/BD-API	Integración de la rama BD-API a main
2026-10-10	eeacab3	Llanazzz	Merge branch 'main' into BD-API	Actualización de la rama BD-API con main

2026-10-10	e28af1e	Llanazzz	Correcion con el GET Y POST /jugadores	API: endpoints de jugadores
2026-10-10	f8b6031	Llanazzz	Agrego ApiCliente con metodos para la API	Clase ApiCliente
2026-10-10	e695004	Llanazzz	Cambio en POST/partidas para coincidir con el cmabio de jugadores	API: endpoint de partidas
2026-10-10	9bad0f2	Llanazzz	Cambio en POST/jugadores para crear o buscar al jugador por nombre	API: crear o buscar jugador por nombre
2026-10-08	57cf40c	Elzymichelle	Merge branch 'ElzyMich' of https://github.com/aleeeebn/uno-interfaces into ElzyMich	Sincronización de la rama ElzyMich
2026-10-08	ac439b3	aleeeebn	Modificaciones al momento de pedir UNO	Reglas: pedir UNO
2026-10-08	1afabff	aleeeebn	Implementación de la pantalla de victoria	Pantalla de victoria
2026-10-07	9a71965	aleeeebn	Merge pull request #12 from aleeeebn/Alex	Integración de la rama Alex a main
2026-10-07	4e5b91a	aleeeebn	Implementacion de botón para pasar de turno	Botón de pasar turno
2026-10-07	dd76630	aleeeebn	Corrección de bug cuando se tenía una única carta	Reglas: una sola carta
2026-10-07	c0ac80c	aleeeebn	Deshabilitacion del boton UNO para cuando no se debe de usar	Reglas: botón UNO
2026-10-07	78ad431	aleeeebn	Mejora y correción de los mensajes de advertencia que superponian ciertos elementos, ahora aparecen abajo del turno actual	Interfaz: mensajes de advertencia
2026-10-07	7eb33cc	aleeeebn	Implementación de diseño. Levantamiento de cartas cuando el jugador de turno actual tiene cartas que ouede jugar	Interfaz: cartas jugables levantadas
2026-10-07	73af817	aleeeebn	Implementación en el diseño de la rueda de dirección de turno	Rueda de dirección de turno
2026-10-07	3a4f4db	aleeeebn	Modificaciones para que el cambio de turno sea estático	Interfaz: cambio de turno
2026-10-06	a887382	Elzymichelle	Merge branch 'main' into ElzyMich	Actualización de la rama ElzyMich con main
2026-10-06	d93b5ef	aleeeebn	Merge pull request #11 from aleeeebn/Alex	Integración de la rama Alex a main
2026-10-05	c241f04	aleeeebn	Arreglos en el robar	Reglas: robar

2026-10-05	5dc584a	Llanazzz	Merge remote-tracking branch 'origin/main' into BD-API	Actualización de la rama BD-API con main
2026-10-05	3c2f0fb	Llanazzz	Agrego Newtonsoft.Json por NuGet	ApiCliente: biblioteca Newtonsoft.Json
2026-10-05	5ba4245	Llanazzz	Ignorar carpeta de NuGet	Git: .gitignore
2026-10-05	0eae8fc	aleeeebn	Merge pull request #10 from aleeeebn/Alex	Integración de la rama Alex a main
2026-10-04	ad8b3af	aleeeebn	Correcciones en el método robar para que sea apegado a las reglas oficiales	Reglas: robar
2026-10-04	07e0e1b	aleeeebn	Implementación de la carta de detrás en la interfaz gráfica	Interfaz: carta de dorso
2026-10-04	fc348bf	aleeeebn	Merge pull request #9 from aleeeebn/Alex	Integración de la rama Alex a main
2026-10-04	f550624	aleeeebn	Arreglo de bugs en la duplicacion de metodos	Corrección de métodos duplicados
2026-10-04	7bcffd6	Elzymichelle	Merge pull request #8 from aleeeebn/ElzyMich	Integración de la rama ElzyMich a main
2026-10-04	cf9a4a9	Elzymichelle	Merge branch 'main' into ElzyMich	Actualización de la rama ElzyMich con main
2026-10-04	e8061bb	Elzymichelle	.	Ajuste menor
2026-10-04	46dc8e9	Elzymichelle	Fin de partida	Form2: fin de partida
2026-10-04	caa890b	Elzymichelle	Botón UNO	Form2: botón UNO
2026-10-04	41f84e7	Elzymichelle	Robar carta	Form2: robar carta
2026-10-04	c796f4e	Elzymichelle	Elegir color tras un comodín	Form2: elegir color
2026-10-04	c3d4627	Elzymichelle	Penalización si se quedó con 1 carta sin decir UNO	Form2: penalización por no decir UNO
2026-10-04	bc5ab04	Elzymichelle	hay ganador?	Form2: comprobar ganador
2026-10-04	0df311c	Elzymichelle	jugar una carta	Form2: jugar una carta

2026-10-04	30335c8	Elzymichelle	ventana del juego	Form2: ventana del juego
2026-10-04	315c8ed	Elzymichelle	dibuja todo el tablero leyendo directamente la partida	Form2: dibujo del tablero
2026-10-04	6068c43	Elzymichelle	Implementar método público para obtener y reescalar imágenes	ImagenesCartas: obtener y reescalar imágenes
2026-10-04	dde03a5	Elzymichelle	Implementar mapeo completo de nombres de archivo de las cartas	ImagenesCartas: nombres de archivo
2026-10-04	7c148e0	Elzymichelle	implementar helper de prefijos de color para nombres de archivo	ImagenesCartas: prefijos de color
2026-10-04	673f2f1	Elzymichelle	agregar método privado para la resolución de ruta de la carpeta assets	ImagenesCartas: ruta de assets
2026-10-04	51a0855	Elzymichelle	inicializar clase estática e infraestructura de caché	ImagenesCartas: caché de imágenes
2026-10-04	c8922f5	Elzymichelle	agregue un nuevo forms que se llama forms 2 para realizar la interfaz sin botones	Form2: nueva interfaz sin botones
2026-10-02	6de9ad8	Llanazzz	Merge pull request #7 from aleeeebn/BD-API	Integración de la rama BD-API a main
2026-10-02	38087aa	Llanazzz	Agrego GET historial	API: GET /historial
2026-10-02	58808c8	Llanazzz	Agrego POST movimientos	API: POST /movimientos
2026-10-02	d0dc078	Llanazzz	Agrego POST /partidas/{id}/terminar	API: terminar partida
2026-10-02	41fa164	Llanazzz	Correcion en las funciones de jugadores y partidas	API: jugadores y partidas
2026-10-02	a602908	Llanazzz	Agrego endpoints y post/partidas	API: endpoint de partidas
2026-10-02	b60101d	Llanazzz	Agrego endpoint jugadores	API: endpoint de jugadores
2026-10-02	5b99331	Llanazzz	Merge branch 'main' into BD-API	Actualización de la rama BD-API con main
2026-10-01	99cb1d2	Elzymichelle	completar interfaz de juego, imágenes de descarte y regla UNO	Form2: interfaz completa y descarte
2026-10-01	c6c0c28	Elzymichelle	agregue el metodo elegir color	Interfaz: elegir color
2026-10-01	c4a6bd0	Elzymichelle	agregue actualiza descarte y elimine el antiguo	Interfaz: pila de descarte

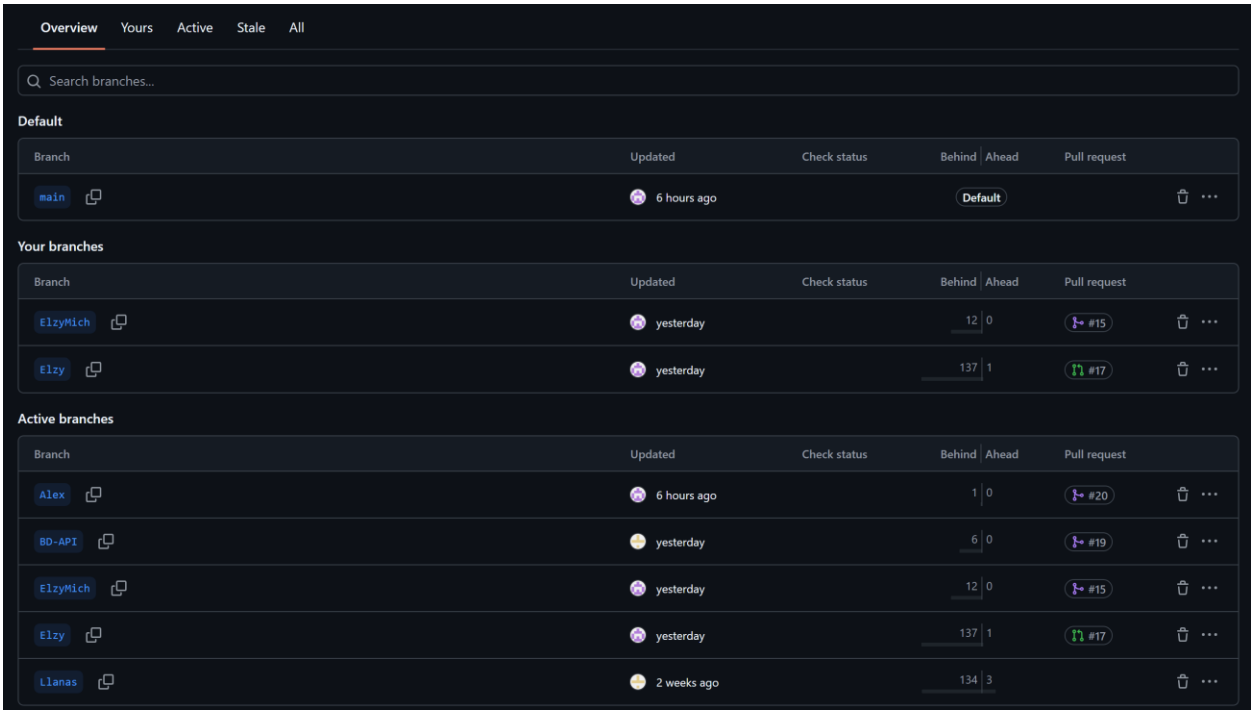
2026-10-01	90e7dc8	Elzymichelle	Usar las imágenes en las manos y opaca las cartas que no se pueden jugar	Interfaz: manos con imágenes
2026-10-01	348f3e5	Elzymichelle	cambie el metodo mostrar jugador actual para ocultar a los jugadores que no están en turno	Interfaz: jugador en turno
2026-10-01	d7cb514	Elzymichelle	uni todo en la clase iamagenes cartas	ImagenesCartas: unificación de la clase
2026-10-01	2d18568	Elzymichelle	agreugue el dibujo programático para la carta dorso	ImagenesCartas: carta de dorso
2026-10-01	e09dfe6	Elzymichelle	agregue el mapeo de nombres de archivo y método Obtener para cartas	ImagenesCartas: nombres de archivo
2026-10-01	e3b18d9	Elzymichelle	agregue la clase estática ImagenesCartas y lógica de caché de imágenes	ImagenesCartas: clase y caché
2026-10-01	066adff	Elzymichelle	agregue una naueva clase llamada imagenes cartas	ImagenesCartas: nueva clase
2026-10-01	97f44e5	aleeeebn	Merge pull request #6 from aleeeebn/Alex	Integración de la rama Alex a main
2026-10-01	b04757a	aleeeebn	assets de cartas creadas	Assets de las cartas
2026-10-01	b89b22d	aleeeebn	Modificacion de tuno	Lógica de turnos
2026-10-01	82c589e	aleeeebn	Cambio	Ajuste menor
2026-10-01	db5b1a0	aleeeebn	Creación de más barajas	Mazo: más barajas
2026-10-01	2cf39ee	Llanazzz	Librerias para utilizar la API en Python	API: bibliotecas de Python
2026-10-01	74f39df	Llanazzz	gitignore con los archivos irrelevantes de python	Git: .gitignore de Python
2026-10-01	32cc5a8	Elzymichelle	.	Ajuste menor
2026-10-01	79ec6da	Elzymichelle	Solución Visual Studio	Proyecto: solución de Visual Studio
2026-10-01	09b34ce	Llanazzz	Merge pull request #5 from aleeeebn/BD-API	Integración de la rama BD-API a main
2026-10-01	1919089	Llanazzz	Agrego diagrama ER para el reporte	Base de datos: diagrama ER
2026-10-01	e5c5670	Llanazzz	Ignoro respaldos de Workbench	Git: .gitignore

2026-10-01	07eccec	Llanazzz	Script SQL y jugadores iniciales	Base de datos: script SQL
2026-09-30	e9e4264	Llanazzz	Agrego llaves foraneas al modelo	Base de datos: llaves foráneas
2026-09-30	e68d83f	Llanazzz	Agrego modelo de BD con las 4 tablas	Base de datos: modelo de 4 tablas
2026-09-30	6e60627	Llanazzz	On BD-API: !!GitHub_Desktop<BD-API>	Git: guardado temporal de GitHub Desktop
2026-09-30	ca11617	Llanazzz	index on BD-API: f0ee22e Merge pull request #4 from aleeeebn/Alex	Git: guardado temporal de GitHub Desktop
2026-09-30	f0ee22e	aleeeebn	Merge pull request #4 from aleeeebn/Alex	Integración de la rama Alex a main
2026-09-30	4f224a7	aleeeebn	Comprobaciones de fuera de index	Lógica: comprobaciones
2026-09-30	f042b36	aleeeebn	Correcciones	Lógica: correcciones
2026-09-30	3babc14	aleeeebn	Comprobación y métodos para victoria	Lógica: victoria
2026-09-30	1961c23	aleeeebn	Comprobación para que no inicie con un comodin el juego	Reglas: comodín inicial
2026-09-30	4254810	aleeeebn	Merge pull request #3 from aleeeebn/Alex	Integración de la rama Alex a main
2026-09-30	c6cea0e	aleeeebn	Creación para comprobar el cambio de color en los comodines	Reglas: cambio de color
2026-09-30	7046e8f	aleeeebn	Cambios para cambiar de color	Reglas: cambio de color
2026-09-30	3c03182	aleeeebn	Métodos para robar y pasar implementados	Reglas: robar y pasar
2026-09-30	41583b3	aleeeebn	Método para mostrar la cantidad de cartas de cada jugador restantes	Lógica: cartas restantes
2026-09-30	5adb552	aleeeebn	Métodos para comprobar que cartas que pueden jugar y cuales no	Reglas: cartas jugables
2026-09-30	2f303c4	aleeeebn	Método para que los jugadores dejen sus cartas en el juego y aparezca en pantalla	Lógica: jugar una carta
2026-09-30	3a820f7	aleeeebn	Diseño primitivo para comprobar los métodos junto con arreglos en el barajear y repartir	Prueba de métodos; barajear y repartir
2026-09-29	0840526	aleeeebn	Comprobaciones de si el usuario tiene cartas	Lógica: manos de los jugadores

2026-09-29	52f7608	aleeeebn	Creación del método para verificar si un mazo está vacio y agregarle cartas desde el mazo jugado	Mazo: rellenar desde el mazo jugado
2026-09-29	dfd2826	aleeeebn	Comprobaciones y cambios respectivo a la lógica	Lógica del juego
2026-09-29	8bf9e62	aleeeebn	Método para barajear	Mazo: barajear
2026-09-29	5b98dc7	aleeeebn	Creación de clase Partida donde administraremos como se jugara y cuales son las reglas, será donde se guardará la mayor parte de los datos	Clase Partida
2026-09-29	d34c49b	aleeeebn	Merge pull request #2 from aleeeebn/Alex	Integración de la rama Alex a main
2026-09-29	18a7507	aleeeebn	Merge remote-tracking branch 'origin/Alex' into Alex	Actualización de la rama Alex
2026-09-29	07f9d12	aleeeebn	.	Ajuste menor
2026-09-29	f98df7b	aleeeebn	Creación de la clase Mazo e inicialización de sus cartas ya defininidas	Clase Mazo
2026-09-29	3000fa9	aleeeebn	setters y getters de carta y definiciones de las cartas	Clase Carta
2026-09-29	b84e07f	aleeeebn	Clase jugador creada con sus setters y getters	Clase Jugador
2026-09-29	da71cc1	aleeeebn	gitignore	Git: .gitignore
2026-09-29	634a774	aleeeebn	ultimo cambio de prueba	Prueba de Git
2026-09-28	3fc78cc	Llanazzz	Cambio de prueba (ignorar)	Prueba de Git
2026-09-28	dbe4c14	Llanazzz	Merge branch 'Llanas' of https://github.com/aleeeebn/uno-interfaces into Llanas	Sincronización de la rama Llanas
2026-09-28	51e1bd2	Llanazzz	Agregar .gitignore y quitar .vs	Git: .gitignore
2026-09-28	28af79e	Llanazzz	Merge pull request #1 from aleeeebn/Llanas	Integración de la rama Llanas a main
2026-09-28	c242267	Llanazzz	Commit de prueba	Prueba de Git
2026-09-24	8e546a3	Llanazzz	Commit de prueba	Prueba de Git
2026-09-24	f3201b6	Llanazzz	lll	Prueba de Git

2026-09-22	6cbc46d	aleeeebn	(no se que sea pero me pide commit)	Prueba de Git
2026-09-22	533af1f	aleeeebn	Plantilla C#	Proyecto base en C#

8. Listado de ramas



9. Problemas encontrados y cómo se solucionaron

Sobre el proyecto general

Problema	Solución
Los movimientos se guardaban desordenados porque las peticiones a la API llegaban en distinto orden que los clics.	Se implementó una cola que manda cada petición después de la anterior, así el log respeta el orden real de las jugadas.
Al cambiar el juego a rondas hubo que redefinir qué es una partida.	Ahora una partida es el juego completo hasta 500 puntos; cada ronda queda en el log como ganar_ronda con sus puntos y solo quien llega a 500 cierra la partida como ganador.
Al fusionar el menú se perdieron las llamadas a la API en Form2.	Se volvieron a colocar las llamadas a la API que guardan jugadores, partida y movimientos.

Sobre el uso de Git y GitHub

Problema	Solución
Las carpetas bin y obj estaban en seguimiento y cada git pull se abortaba.	Se resolvió con git restore y sacando esas carpetas del seguimiento, además de agregarlas al .gitignore.
Un merge sin conflicto dejó métodos duplicados en Partida.cs y el proyecto no compilaba.	Se borró el método duplicado. Desde entonces se compila el proyecto después de cada merge para detectar errores a tiempo.
Había archivos que no debían subirse (entorno virtual, credenciales, respaldos de Workbench, paquetes de NuGet).	Se mantuvo un .gitignore con el entorno virtual, el archivo .env, los respaldos de Workbench y los paquetes de NuGet.

Aprendizaje principal: un merge sin conflictos no garantiza que el código funcione, por eso conviene compilar y probar el juego después de integrar ramas.

10. comandos

```
En Workbench primero database/uno_db.sql
En la carpeta api, un comando a la vez:
python -m venv .venv
venv\Scripts\activate
pip install -r requirements.txt
Copien .env.example como .env y pongan su contraseña de MySQL en DB_PASSWORD
(el .env no se sube al repo)
Levántenla desde la carpeta api:
uvicorn main:app --reload
```

11. Conclusiones

Se logró un UNO completo y funcional, con interfaz gráfica, reglas oficiales, rondas con puntos y conexión a una base de datos que guarda jugadores, historial y el log de cada movimiento.

El reparto en tres áreas (interfaz, lógica y base de datos con API) permitió que los tres integrantes programaran en paralelo, cada quien en su rama. Los problemas con Git, como las carpetas bin y obj en seguimiento o los métodos duplicados tras un merge, enseñaron a mantener un buen. gitignore y a compilar después de cada integración.